Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет имени И. Н. Ульянова»

Кафедра компьютерных технологий

# КРОССПЛАТФОРМЕННОЕ ПРИЛОЖЕНИЕ «ПЛАНИРОВЩИК СЕТЕЙ И МАРШРУТОВ»

**Технический проект**

Лабораторная работа № 3
по дисциплине «Технология разработки программного обеспечения»

Выполнил: студент группы КТ-41-24
Егоров Артемий Сергеевич

Проверил: Мытников А. Н.

Чебоксары, 2026

**Содержание**

1 Структурная схема . . . . . 3
2 Функциональная схема . . . . . 3
3 Разработка алгоритмов . . . . . 4
3.1 Уровень 1. Основной сценарий . . . . . 4
3.2 Уровень 2. Детализация расчетных операций . . . . . 5
3.2.1 Импорт дорожной сети . . . . . 5
3.2.2 Выбор новых соединений по алгоритму Краскала . . . . . 5
3.2.3 Поиск кратчайшего пути алгоритмом A* . . . . . 6
3.2.4 Расчет замкнутого маршрута . . . . . 6
3.2.5 Сохранение и восстановление проекта . . . . . 7
3.3 Структуры данных и технические ограничения . . . . . 7
4 Структурная карта Константайна . . . . . 9
5 Структурная карта Джексона . . . . . 10

## 1 Структурная схема

Приложение «Планировщик сетей и маршрутов» разрабатывается на Kotlin Multiplatform. Логика проекта, представление дорожного графа и алгоритмы расчета размещаются в общем модуле. Пользовательский интерфейс реализуется на Compose Multiplatform. Доступ к файлам и картографическому сервису осуществляется через платформенные адаптеры.

Структурная схема на Рисунок 1 разделяет пользовательский интерфейс, общую логику и операции, зависящие от операционной системы. Модуль расчетов не должен обращаться к элементам интерфейса или картографическому сервису. Он получает подготовленный граф и параметры задачи.

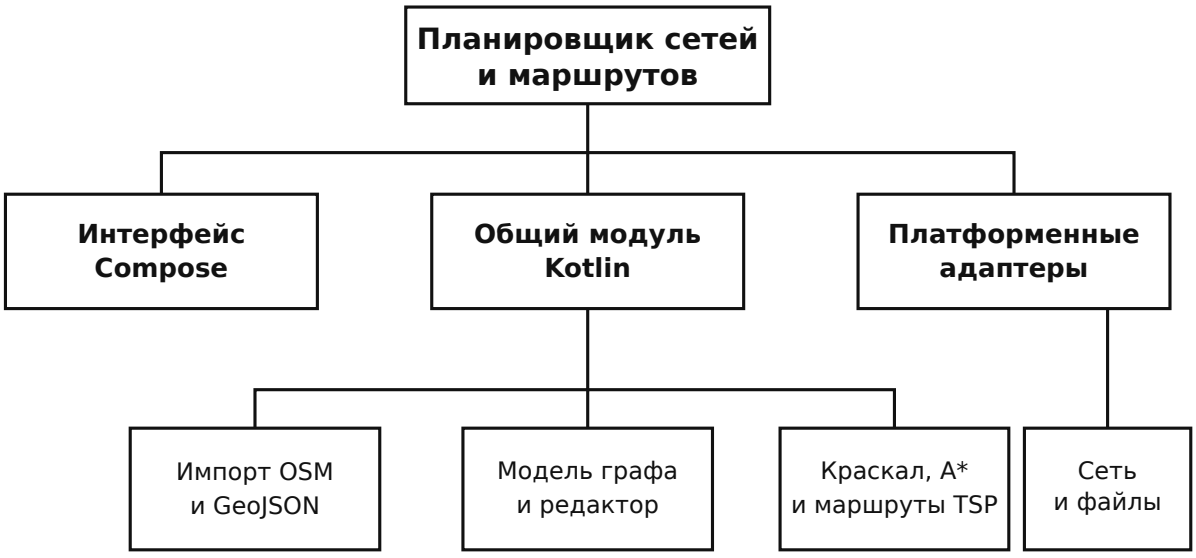



Рисунок 1. Структурная схема приложения

Данные проекта проходят через модель графа. Модуль импорта создает исходные вершины и ребра, редактор применяет изменения пользователя, а модуль расчетов получает выбранный вариант сети. Сохранение и восстановление выполняются через общий интерфейс файлового доступа.

## 2 Функциональная схема

Функциональная схема на Рисунок 2 показывает передачу данных между операциями. Источником существующих дорог служат файлы OSM XML, GeoJSON либо картографический сервис. После импорта получается граф, который можно дополнить проектируемыми участками. По одному варианту графа выполняются две расчетные задачи: выбор новых соединений по стоимости и маршрутизация по длинам доступных ребер.

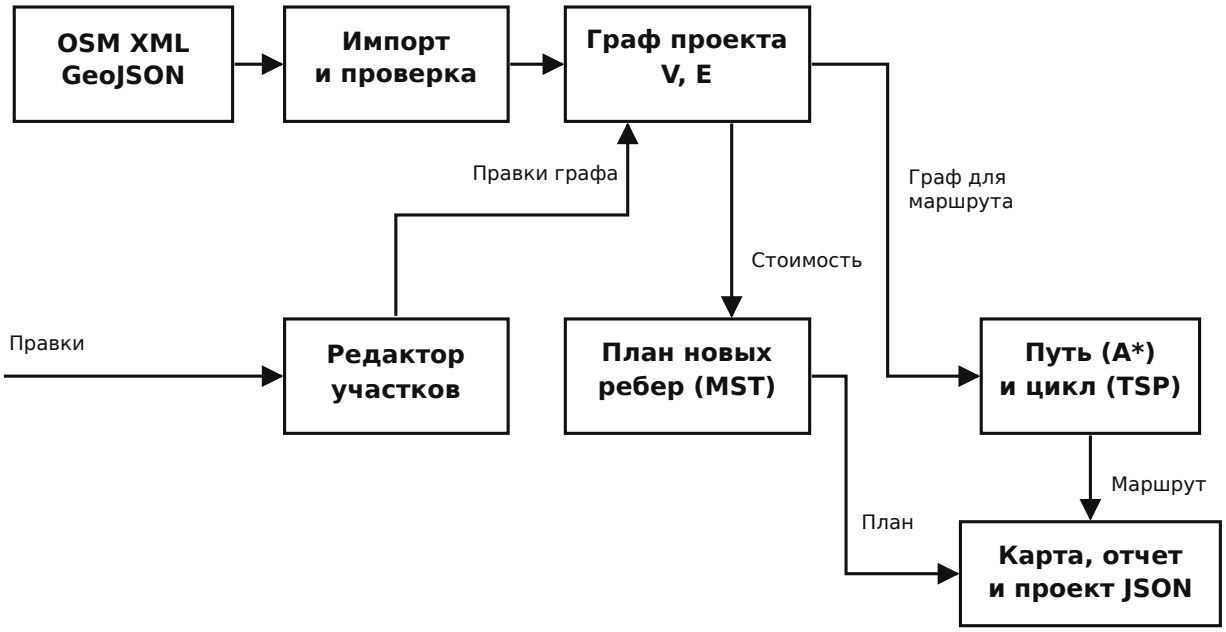



Рисунок 2. Функциональная схема с основными потоками данных

Расчет плана строительства использует стоимость проектируемых ребер, а поиск пути использует длины ребер и ограничения движения. Оба результата остаются отдельными данными проекта. Пользователь может принять предложенные новые участки в расчетный вариант, после чего выполнить поиск маршрута уже по этому варианту сети.

## 3 Разработка алгоритмов

### 3.1 Уровень 1. Основной сценарий

Метод пошаговой детализации применяется к действиям пользователя и к расчетным операциям. На первом уровне определен общий порядок работы приложения.

1. При запуске показать список сохраненных проектов и команду создания нового проекта.
2. Загрузить выбранный проект либо создать пустую схему территории.
3. По команде пользователя импортировать дорожную сеть из карты или файла.
4. Предоставить редактирование вершин, существующих и проектируемых участков.
5. Выполнить выбранную расчетную операцию: план строительства, кратчайший путь или замкнутый маршрут.
6. Отобразить результат на схеме и в текстовом виде.

7. При изменении проекта пометить предыдущие результаты как требующие пересчета.
8. Сохранить проект и завершить работу по команде пользователя.

Импорт данных, редактирование и расчеты могут выполняться многократно в рамках одного проекта. При расчете запрещается незаметно изменять исходные данные или автоматически добавлять предложенные к строительству участки в активный вариант сети.

## 3.2 Уровень 2. Детализация расчетных операций

### 3.2.1 Импорт дорожной сети

**Вход:** область карты либо файл OSM XML или GeoJSON. **Выход:** исходный граф с вершинами, ребрами и характеристиками путей.

1. Получить данные и проверить возможность их чтения.
2. Извлечь геометрию дорог и пешеходных путей, их идентификаторы и признаки доступности.
3. Разбить линии на прямолинейные участки между последовательными географическими точками.
4. Определить действительные соединения участков, сохраняя сведения об одностороннем движении и разрешенных способах перемещения.
5. Вычислить длину каждого ребра по координатам его концов в метрах.
6. Проверить уникальность идентификаторов и наличие вершин у каждого ребра.
7. Сохранить импортированную сеть и сведения об источнике в проекте. Пользовательские изменения хранить отдельно.

При ошибке импорта текущий проект не заменяется частично прочитанными данными. Изменения применяются только после успешной проверки сформированного графа.

### 3.2.2 Выбор новых соединений по алгоритму Краскала

**Вход:** выбранные точки, существующие соединения, допустимые новые участки и стоимость строительства каждого нового участка. **Выход:** список предлагаемых участков и суммарная стоимость.

1. Проверить, что все выбранные точки существуют, а стоимость новых соединений неотрицательна.
2. Создать структуру непересекающихся множеств (DSU) для компонент связности.
3. Объединить вершины, уже соединенные существующими дорогами: дополнительная стоимость таких связей равна нулю.

4. Упорядочить допустимые новые участки по стоимости, затем по идентификатору для однозначности результата.
5. Последовательно рассматривать новые участки. Если концы участка принадлежат разным компонентам, включить его в план и объединить компоненты.
6. Остановиться, когда все требуемые точки оказались связаны, либо закончились допустимые участки.
7. При успехе вычислить сумму стоимости выбранных новых ребер; при неудаче сообщить, какие точки связать не удалось.

Краскал выбирает минимальный набор ребер среди допустимых кандидатов для связи заданных компонент сети. Программа не должна выдавать этот результат за расчет трасс, которых пользователь не задавал, или за полноценную инженерную смету строительства.

### 3.2.3 Поиск кратчайшего пути алгоритмом A*

**Вход:** расчетный вариант сети, начальная и конечная вершины, автомобильный или пешеходный режим. **Выход:** последовательность ребер пути и его длина либо сообщение об отсутствии пути.

1. Проверить существование начальной и конечной вершин.
2. Отобрать участки, доступные выбранному способу перемещения, и учитывать направление каждого ребра.
3. Для начальной вершины установить накопленную длину равной нулю; остальные оценки считать бесконечными.
4. Использовать очередь с приоритетом по сумме пройденной длины и эвристики расстояния по прямой до цели.
5. Последовательно извлекать вершину с наименьшей оценкой и улучшать длины путей до допустимых соседей.
6. При достижении конечной вершины восстановить путь по сохраненным предшественникам.
7. Если очередь опустела раньше, вывести сообщение об отсутствии пути.

Поскольку ребра представляют прямолинейные участки, их длина не меньше расстояния по прямой между концами. Поэтому евклидова эвристика не завышает оставшуюся длину пути.

### 3.2.4 Расчет замкнутого маршрута

**Вход:** расчетный граф, начальная точка и список мест посещения. **Выход:** порядок посещения, полный путь по ребрам графа, общая длина и признак точного или приближенного решения.

1. Проверить, что все точки принадлежат графу и число мест посещения не превышает 50.

2. Для каждой пары выбранных точек найти кратчайший допустимый путь алгоритмом A* и записать его длину в матрицу расстояний.
3. Если необходимого перехода нет, учесть недостижимость при построении цикла. Если замкнутый обход невозможен, сообщить об этом.
4. При числе мест посещения не более 12 применить динамическое программирование Хелда-Карпа.
5. При числе мест посещения от 13 до 50 построить начальный маршрут методом ближайшего соседа и улучшить его перестановками 2-opt с пересчетом направленных переходов.
6. Восстановить полный маршрут из найденных ранее кратчайших путей между последовательными точками посещения, включая возврат на старт.
7. Вывести порядок точек, длину и признак точности. Для приближенного результата не утверждать глобальную оптимальность.

Поиск пути и расчет замкнутого маршрута работают с длиной дороги, а не со стоимостью строительства. Новые соединения участвуют в этих расчетах только после включения пользователем в выбранный вариант сети.

### 3.2.5 Сохранение и восстановление проекта

1. Подготовить снимок текущего состояния проекта с номером версии формата.
2. Сериализовать вершины, ребра, варианты сети, параметры маршрутов и источник картографических данных в JSON UTF-8.
3. Записать данные во временный файл и завершить запись заменой основного файла при успешном сохранении.
4. При открытии проверить формат, обязательные поля, уникальность идентификаторов и ссылки между объектами.
5. При ошибке проверки не изменять открытый проект и показать причину отказа.

## 3.3 Структуры данных и технические ограничения

Данные проекта разделены на исходную карту, изменения пользователя, варианты сети и результаты расчетов. Такое разделение позволяет обновлять импортированную карту и повторять расчеты без потери внесенных вручную участков.

| **Объект** | **Основные поля** |
|---|---|
| Проект | Идентификатор, версия формата, режим перемещения, источник и дата импорта. |
| Вершина | Уникальный идентификатор, координаты, имя и тип точки. |

| | |
|---|---|
| Ребро | Идентификатор, концы, длина, статус, разрешенное направление и режим движения, стоимость нового участка. |
| Вариант сети | Идентификатор, выбранные новые участки, список точек посещения и начальная точка. |
| Результат | Выбранные ребра строительства, стоимость, последовательность маршрута, длина, признак точности. |

Техническое задание предусматривает обработку проекта объемом до 20 000 вершин и 50 000 ребер, а также маршрут до 50 точек посещения.

Пользователь работает с проектом локально. Сетевой обмен выполняется при запросе картографических данных, а сохранение, редактирование и расчет по уже загруженному графу доступны без Интернета.

## 4 Структурная карта Константайна

Структурная карта Константайна (Рисунок 3) показывает декомпозицию обработки проекта. Координатор вызывает модуль подготовки сети, расчетный модуль или модуль хранения по выбранной команде. Подготовка разделена на импорт и редактирование; расчеты разделены на построение новых соединений, поиск пути и замкнутый обход. Чтение проекта и его сохранение выполняют разные модули.

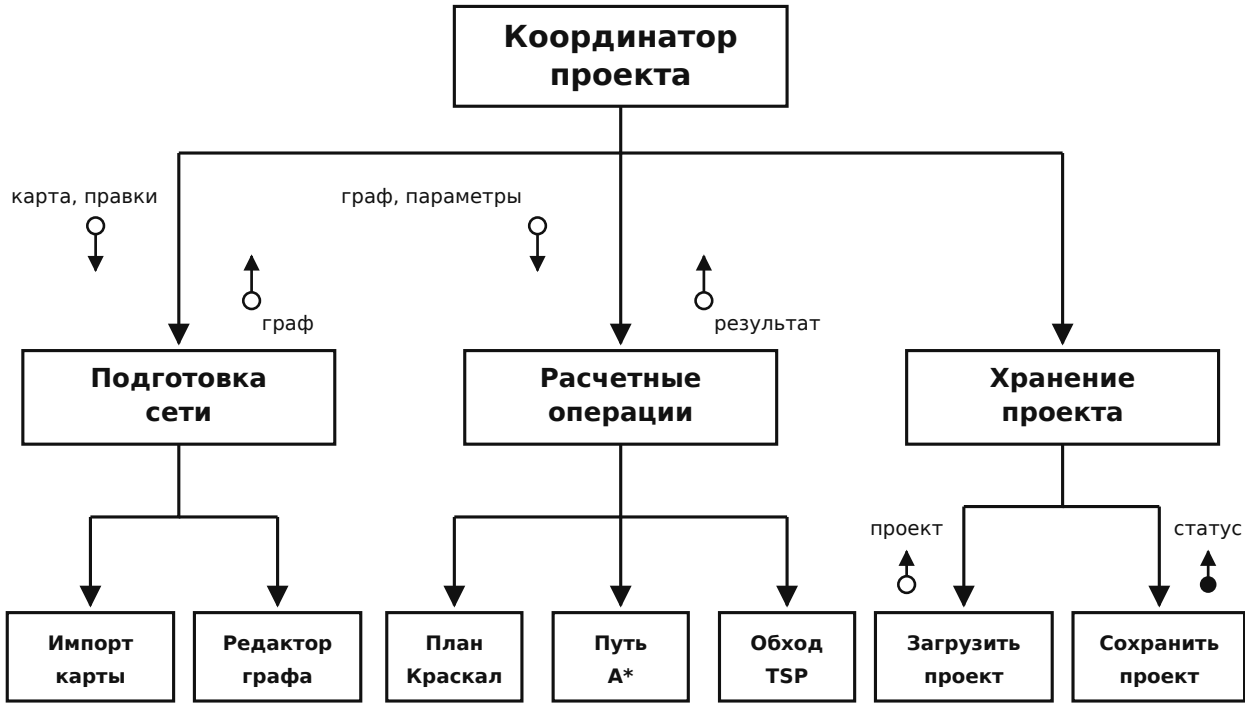



Рисунок 3. Структурная карта Константайна

Импорт загружает существующие дороги из картографических данных, а редактор применяет изменения пользователя к выбранному варианту сети. Расчетный модуль передает отдельным алгоритмам граф и параметры задачи, после чего возвращает план новых соединений или маршрут. Загрузка возвращает координатору данные прочитанного проекта, сохранение возвращает статус записи. Обе операции используют JSON и платформенный файловый адаптер. Расчеты не изменяют граф, а возвращают результат координатору.

## 5 Структурная карта Джексона

Карта Джексона на Рисунок 4 описывает порядок действий в одном сеансе. Сначала пользователь открывает или создает проект, затем многократно выбирает команду и в конце завершает работу. При выборе расчета выполняется один из трех алгоритмов.

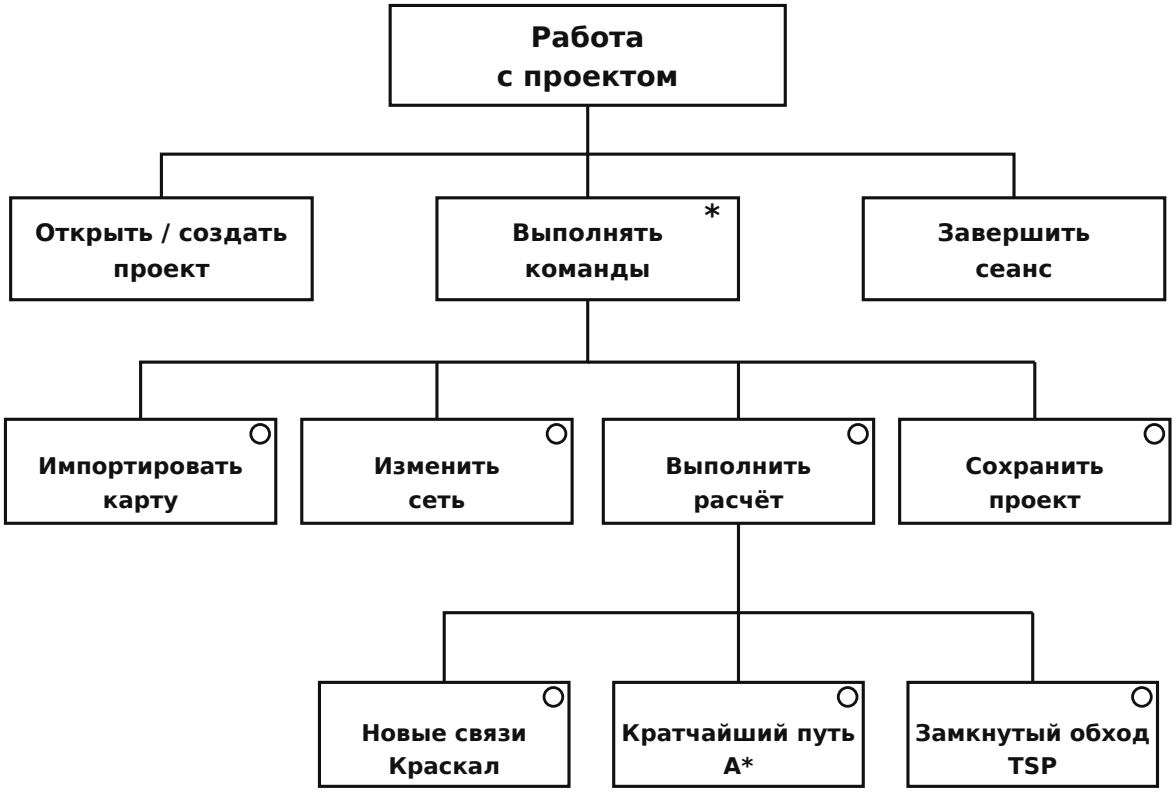



Рисунок 4. Структурная карта Джексона